Løsningsforslag

Oppgave 4.1a

Se vedlagt fil 'lokallag.xml'. NB! I oppgaven står det at man skal lage en XML som "representerer et lokallag". Derfor blir det riktig at er rotnoden, og at er barnenoder. Men har man også lagt opp til en ekstra rotnode, f.eks. , så er det også helt greit (og kanskje naturlig).

Oppgave 4.1b

Se vedlagt fil 'lokallag.xsd'. Se også kobling mellom XML og XSD i 'lokallag.xml' (xsi:noNamespaceSchemaLocation). NB! Her kan man også bruke (xsi:schemaLocation), men det er ikke nødvendig.

Oppgave 4.1c

```
SELECT
   XMLELEMENT(
      NAME "aktive-medlemmer",
      XMLAGG(
         XMLELEMENT(
            NAME "medlem",
            XMLATTRIBUTES(m.medlemsnummer AS "mId"),
            XMLELEMENT(NAME "fornavn", m.fornavn),
            XMLELEMENT(NAME "etternavn", m.etternavn),
            CASE WHEN a.betalt = 'J' THEN
               XMLELEMENT(
                  NAME "avgift-2026",
                  XMLATTRIBUTES('J' AS "betalt", a.betalingsdato AS
"betalingsdato"))
                                          ELSE
               XMLELEMENT(
                  NAME "avgift-2026",
                  XMLATTRIBUTES('N' AS "betalt"))
            END
         )
         ORDER BY m.medlemsnummer
      )
   )
FROM  medlem m INNER JOIN medlemsavgift a ON (m.medlemsnummer =
a.medlemsnummer)
WHERE m.aktiv = true
  AND m.lokallag_id = 1
  AND a.ar = 2026;
```

Opgave 4.2a

Se vedlagt fil 'lokallag.json'. NB! Det er ikke nødvendig å ha en ekstra nøkkel-verdi par for "lokallag". Filen heter "lokallag.json", så det er naturlig at det er en liste med medlemmer direkte i rotnoden. Men man får riktig også hvis man for eksempel har lagt opp til at det kan være flere lokallag:

```
{
   "lokallag": [
      {
        "id": 1,
        "navn": "Oslo",
        ...
        "medlemmer": [
            {
               "mId": 1,
               "fornavn": "Ola",
               "etternavn": "Nordmann",
               "avgift-2026": {
                  "betalt": "J",
                  "betalingsdato": "2026-01-15"}
            },
            {
               "mId": 2,
               "fornavn": "Kari",
               "etternavn": "Nordmann",
               "avgift-2026": {
                  "betalt": "N"
               }
            }
        ]
      },
     {
        "id": 2,
        "navn": "Bergen Fiskerlag",
        ...
        "medlemmer": [
          {
            "mId": 3,
            "fornavn": "Per",
            "etternavn": "Fisker",
            "avgift-2026": {
              "betalt": "J",
              "betalingsdato": "2026-02-20"
            }
          }
        ]
     }
   ]
}
```

Oppgave 4.2b

```
SELECT json_build_object(
             'aktive_medlemmer',
             json_agg(
                   json_build_object(
                         'medlemsnummer', m.medlemsnummer,
                         'fornavn', m.fornavn,
                         'etternavn', m.etternavn,
                         'avgift_2026', json_build_object(
                               'betalt', a.betalt,
                               'betalingsdato', CASE WHEN a.betalt = 'J' THEN
                                                          a.betalingsdato END
                                       )
                   )
                   ORDER BY m.medlemsnummer
             )
       )
FROM medlem m INNER JOIN medlemsavgift a ON (m.medlemsnummer =
a.medlemsnummer)
WHERE m.aktiv = true
  AND m.lokallag_id = 1
  AND a.ar = 2026;
```

Oppgave 4.3a

Svar: MongoDB

Begrunnelse:

MongoDB er en dokumentdatabase som passer godt fordi: Fleksibel datamodell – medlemmer, arrangementer og påmeldinger har ulik struktur og kan modelleres naturlig som JSON-dokumenter Embedded data – relatert data (f.eks. medlemsavgifter per medlem) kan lagres sammen i ett dokument, noe som gir raske oppslag uten joins Godt egnet for web-applikasjoner – MongoDB har god støtte for CRUD-operasjoner og skalerer horisontalt

Hvorfor ikke de andre?

Neo4j (graf) – best egnet når relasjoner mellom data er det viktigste (sosiale nettverk, anbefalingsmotorer). Her er det ikke komplekse relasjoner mellom medlemmer.

Cassandra (kolonnebasert) – best for svært store datamengder med høy skrivefrekvens (logging, IoT). En liten forening har ikke dette behovet.

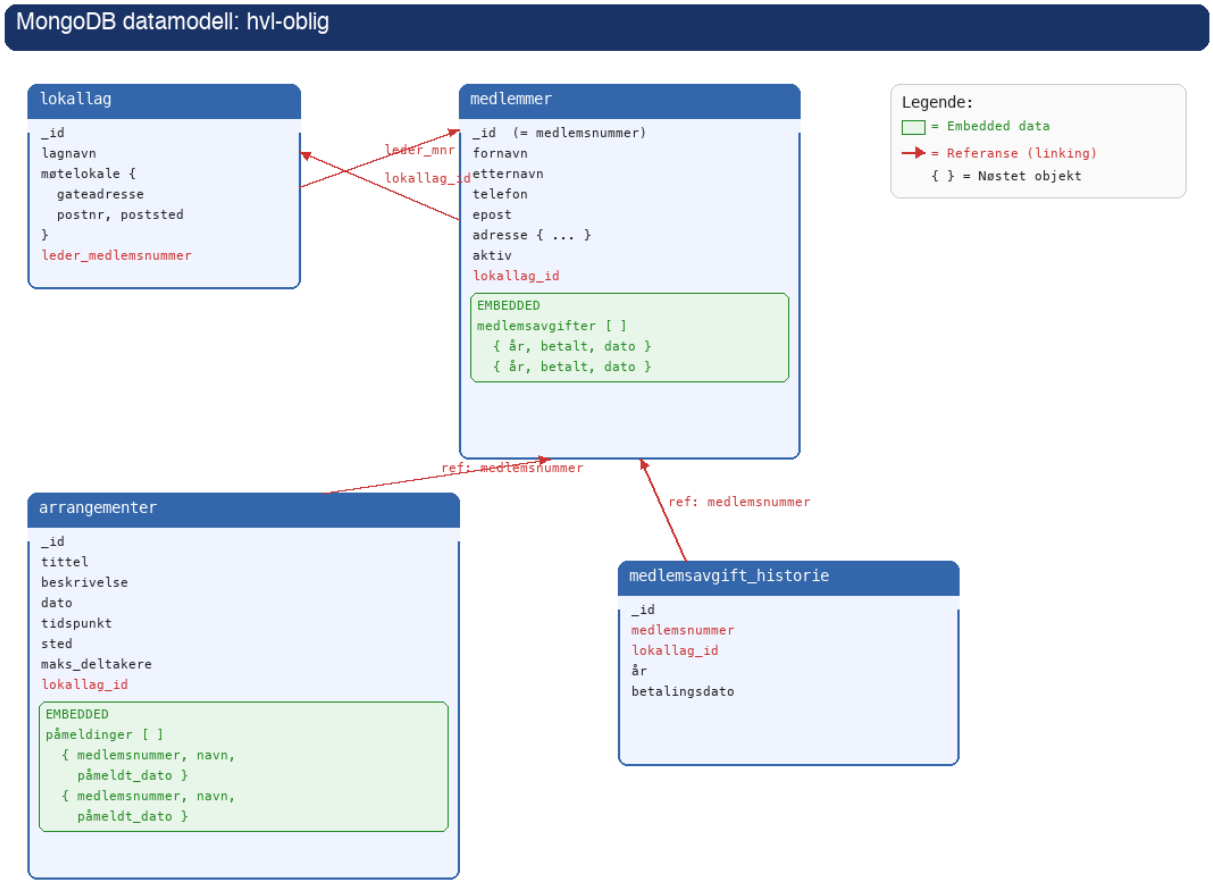
Redis (nøkkel-verdi) – best som cache eller for enkle oppslag. For dårlig støtte for komplekse datastrukturer og spørringer (søk, filtrering, aggregering).

Oppgave 4.3b

Her har jeg valgt å bruke en kombinasjon av embedded documents og referanser for å modellere dataene i MongoDB. Medlemmer og deres (3 siste) medlemsavgifter er modellert som embedded documents, siden de er nært knyttet sammen og ofte vil bli hentet samtidig.

Påmeldinger er modellert som embedded documents i arrangementer, siden de er spesifikke for hvert arrangement og det er naturlig å hente dem sammen med arrangementet. I tilleggs ser vi at noe av medlemsinformasjonen (medlemsnummer og navn) er embeddet i påmeldinger for å unngå behovet for å gjøre ekstra oppslag i medlemmer-samlingen når vi henter arrangementer og deres påmeldinger. Her kunne vi vurdert å ta med enda mere medlemsinformasjon i påmeldinger (hvis man ser behov for dette).

Foreslått Datamodell:



Oppgave 4.3c

Query

MongoDB Query API kommandoer for å sette inn data i “arrangementer”-samlingen:

```
// Koble til databasen
use hvl-oblig

// Opprett samlingen "arrangementer" med 2 arrangementer og 3 påmeldinger hver
db.arrangementer.insertMany([
    {
        _id: 1,
        tittel: "Brettspillkveld – Catan-turnering",
        beskrivelse: "Åpen turnering i Settlers of Catan for alle medlemmer",
        dato: "2026-05-10",
        tidspunkt: "18:00",
        sted: "Strandgaten 15, Bergen",
        maks_deltakere: 20,
        lokallag_id: 1,
        påmeldinger: [
            { medlemsnummer: 1001, navn: "Lars Jenssen",  påmeldt_dato: "2026-04-20" },
            { medlemsnummer: 1003, navn: "Ola Nordmann",  påmeldt_dato: "2026-04-21" },
            { medlemsnummer: 1005, navn: "Jonas Berg",    påmeldt_dato: "2026-04-22" }
        ]
    },
    {
        _id: 2,
        tittel: "Åpen spillkveld – Ticket to Ride",
        beskrivelse: "Uformell spillkveld med Ticket to Ride Europe",
        dato: "2026-05-24",
        tidspunkt: "17:30",
        sted: "Strandgaten 15, Bergen",
        maks_deltakere: 16,
        lokallag_id: 1,
        påmeldinger: [
            { medlemsnummer: 1002, navn: "Kari Hansen",   påmeldt_dato: "2026-05-01" },
            { medlemsnummer: 1004, navn: "Emma Nilsen",   påmeldt_dato: "2026-05-02" },
            { medlemsnummer: 1001, navn: "Lars Jenssen",  påmeldt_dato: "2026-05-03" }
        ]
    }
])
```

Screenshots (som viser at data er satt inn)

Følgende bilder viser at dataene for de to arrangementene og deres påmeldinger er korrekt satt inn i MongoDB.

```
test> use hvl-oblig
switched to db hvl-oblig
hvl-oblig> show collections
arrangementer
lokallag
medlemmer
medlemsavgift_historie
```

Bilde 1: Liste av samlinger

```
hvl-oblig> db.arrangementer.find({_id:1})
[
  {
    _id: 1,
    tittel: 'Brettspillkveld - Catan-turnering',
    beskrivelse: 'Åpen turnering i Settlers of Catan for alle medlemmer',
    dato: '2026-05-10',
    tidspunkt: '18:00',
    sted: 'Strandgaten 15, Bergen',
    maks_deltakere: 20,
    lokallag_id: 1,
    'påmeldinger': [
      {
        medlemsnummer: 1001,
        navn: 'Lars Jenssen',
        'påmeldt_dato': '2026-04-20'
      },
      {
        medlemsnummer: 1003,
        navn: 'Ola Nordmann',
        'påmeldt_dato': '2026-04-21'
      },
      {
        medlemsnummer: 1005,
        navn: 'Jonas Berg',
        'påmeldt_dato': '2026-04-22'
      }
    ]
  }
]
```

Bilde 2: Dokument for arrangement 1

```
hvl-oblig> db.arrangementer.find({_id:2})
[
  {
    _id: 2,
    tittel: 'Åpen spillkveld - Ticket to Ride',
    beskrivelse: 'Uformell spillkveld med Ticket to Ride Europe',
    dato: '2026-05-24',
    tidspunkt: '17:30',
    sted: 'Strandgaten 15, Bergen',
    maks_deltakere: 16,
    lokallag_id: 1,
    'påmeldinger': [
      {
        medlemsnummer: 1002,
        navn: 'Kari Hansen',
        'påmeldt_dato': '2026-05-01'
      },
      {
        medlemsnummer: 1004,
        navn: 'Emma Nilsen',
        'påmeldt_dato': '2026-05-02'
      },
      {
        medlemsnummer: 1001,
        navn: 'Lars Jenssen',
        'påmeldt_dato': '2026-05-03'
      }
    ]
  }
]
```

Bilde 3: Dokument for arrangement 2

Oppgave 4.4 – MongoDB Query API

a) Legg til et nytt medlem i lokallag 1 (_id=1)

```
db.medlemmer.insertOne({
    _id: 1031,
    fornavn: "Hanne",
    etternavn: "Fredriksen",
    telefon: "11335577",
    epost: "hanne.fredriksen@example.com",
    adresse: {
        gateadresse: "Fjøsangerveien 10",
        postnr: "5054",
        poststed: "Bergen"
    },
    aktiv: true,
    lokallag_id: 1,
    medlemsavgifter: [
        { år: 2025, betalt: "J", betalingsdato: "2025-02-10" },
        { år: 2026, betalt: "N" }
    ]
})
```

b) Legge til 2 nye lokallag

```
db.lokallag.insertMany([
    {
        _id: 2,
        lagnavn: "Stavanger Brettspillklubb",
        møtelokale: {
            gateadresse: "Klubbgata 5",
            postnr: "4013",
            poststed: "Stavanger"
        },
        leder_medlemsnummer: null
    },
    {
        _id: 3,
        lagnavn: "Trondheim Brettspillklubb",
        møtelokale: {
            gateadresse: "Olav Tryggvasons gate 20",
            postnr: "7011",
            poststed: "Trondheim"
        },
        leder_medlemsnummer: null
    }
])
```

c) Slette lokallag 3

```
db.lokallag.deleteOne({ _id: 3 })
```

d) Finn alle lokallagene

```
db.lokallag.find()
```

e) Finn alle medlemmer i lokallag 1

```
db.medlemmer.find({ lokallag_id: 1 })
```

f) Finn de 3 medlemmene med lavest medlemsnummer i lokallag 1

```
db.medlemmer.find({ lokallag_id: 1 }).sort({ _id: 1 }).limit(3)
```

g) Finn alle medlemmer som bor på et bestemt poststed (f.eks. Bergen)

```
db.medlemmer.find({ "adresse.poststed": "Bergen" })
```

h) Finn alle medlemmer som er aktive og har betalt avgift for 2025

Vanskelig! Så her godkjenner vi det meste.

Alternativ 1 (bruker $elemMatch som jeg ikke har gått gjennom i forelesning og/eller videoer):

```
db.medlemmer.find({
    aktiv: true,
    medlemsavgifter: { $elemMatch: { år: 2025, betalingsdato: { $exists: true, $ne: null } } }
})
```

Alternativ 2 - Aggregagate (dette har de engentlig ikke lært):

```
db.medlemmer.aggregate([
    { $match: { aktiv: true } },
    { $unwind: "$medlemsavgifter" },
    { $match: { "medlemsavgifter.år": 2025, "medlemsavgifter.betalingsdato": { $ne: null } } }
])
```

Alternative som kan se ut som er riktig, men som vil være feil (fordi det ikke tar hensyn til at medlemsavgifter er en liste, og at betalingsdato sjekkes uavhengig av år).

```
db.medlemmer.find({
    aktiv: true,
    "medlemsavgifter.år": 2025,
    "medlemsavgifter.betalingsdato": { $exists: true, $ne: null }
})
```

i) Finn alle med fornavn som starter på "A" eller etternavn som starter på "J"

```
db.medlemmer.find({
    $or: [
        { fornavn: { $regex: "^A" , $options: "i" } },
        { etternavn: { $regex: "^J" , $options: "i" } }
    ]
})

Eller:
db.medlemmer.find({
    $or: [
        { fornavn: /^A/i },
        { etternavn: /^J/i }
    ]
})
```

j) Finn antall medlemmer i lokallag 1

```
db.medlemmer.countDocuments({ lokallag_id: 1 })
```

k) (valgfri) Finn antall medlemmer for hvert lokallag

```
Dette har de egentlig ikke lært, men en løsning er å kjøre følgende for hvert lokallag:

db.medlemmer.countDocuments({ lokallag_id: 1 }))

Alternativt:

db.medlemmer.aggregate([
    { $group: { _id: "$lokallag_id", antall: { $sum: 1 } } }
])
```

l) (valgfri) Legge til betalingsinformasjon for 2027 for alle i lokallag 1

```
Studentene har ikke lært å bruke $push (se løsning under).
De har lært at de kan skrive følgende, MEN denne vil erstatte hele “meldemsavgifter” nøkkelen,
og informasjon om de forrige årene forsvinner.

db.medlemmer.updateOne(
  { lokallag_id: 1 },
  { $set: { medlemsavgifter: { år: 2027, betalt: "N" } } }
)



Løsning med $push:

db.medlemmer.updateMany(
    { lokallag_id: 1 },
    { $push: { medlemsavgifter: { år: 2027, betalt: "N" } } }
)
```

Hvis man virkelig skulle løst denne, så var det vel sagt at det skulle være en begrensning på 3 år i medlemsavgifter array. Dette kunne man oppnådd med å bruke $each:

```
db.medlemmer.updateMany(
    { lokallag_id: 1 },
    { $push: {
        medlemsavgifter: {
            $each: [{ år: 2027 }],
            $sort: { år: -1 },
            $slice: 3
        }
    }}
)
```